

PAYMENT ENTITY – SQL DATABASE REPORT

-ABINAYA.R(E25AI102)

SQL Implementation, Constraints and Payment Transaction Analysis

1. Objective

To create and manage a Payment Entity using SQL by implementing table creation, constraints, payment records, and queries for successful, failed and pending transactions.

2. Payment Entity

Attribute	Description
Payment_ID	Unique payment identification
Order_ID	Related order identification
Payment_Date	Date of payment
Payment_Mode	UPI, Card, Cash, etc.
Payment_Status	Success, Failed or Pending
Transaction_Amount	Amount paid

3. Create Payment Table

CREATE TABLE Payment (Payment_ID INT, Order_ID INT, Payment_Date DATE, Payment_Mode VARCHAR(20), Payment_Status VARCHAR(20), Transaction_Amount DECIMAL(10,2));

```
SQL> CREATE TABLE Payment(PAYMENT_ID VARCHAR2(10),ORDER_ID VARCHAR2(7),PAYMENT_DATE DATE,PAYMENT_MODE VARCHAR2(20),PAYMENT_STATUS VARCHAR2(20),TRANSACTION_
AMOUNT NUMBER(10,2));
Table created.
```

4. Constraints

Primary Key

ALTER TABLE Payment ADD PRIMARY KEY (Payment_ID);

```
SQL> ALTER TABLE Payment ADD CONSTRAINT PK_PAYMENT PRIMARY KEY(PAYMENT_ID);
Table altered.
```

Foreign Key

ALTER TABLE Payment ADD CONSTRAINT fk_order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);

```
SQL> ALTER TABLE Payment ADD CONSTRAINT FK_PAYMENT_ORDER FOREIGN KEY(ORDER_ID) REFERENCES Orders(ORDER_ID);
Table altered.
```

Check Constraint

ALTER TABLE Payment ADD CONSTRAINT chk_amount CHECK (Transaction_Amount > 0);

```
SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_PAYMENT_AMOUNT CHECK(TRANSACTION_AMOUNT>0);
Table altered.
```

Not NULL Constraint

```
ALTER TABLE Payment MODIFY(PAYMENT_MODE NOT NULL,PAYMENT_STATUS NOT NULL);
```

```
SQL> ALTER TABLE Payment MODIFY(PAYMENT_MODE NOT NULL,PAYMENT_STATUS NOT NULL);  
Table altered.
```

5. Insert Payment Records

```
INSERT INTO Payment VALUES (1, '0101', '20-SEP-2026', 'UPI', 'Success', 2500);  
INSERT INTO Payment VALUES (2, '0102', '21-SEP-2026', 'Card', 'Success', 1800);  
INSERT INTO Payment VALUES (3, '0103', '22-SEP-2026', 'Cash', 'Pending', 3200);  
INSERT INTO Payment VALUES (4, '0104', '23-SEP-2026', 'UPI', 'Success', 1500);  
INSERT INTO Payment VALUES (5, '0105', '24-SEP-2026', 'Card', 'Failed', 2750);
```

```
SQL> INSERT INTO Payment VALUES (1, '0101', '20-SEP-2026', 'UPI', 'Success', 2500);  
1 row created.  
  
SQL> INSERT INTO Payment VALUES (2, '0102', '21-SEP-2026', 'Card', 'Success', 1800);  
1 row created.  
  
SQL> INSERT INTO Payment VALUES (3, '0103', '22-SEP-2026', 'Cash', 'Pending', 3200);  
1 row created.  
  
SQL> INSERT INTO Payment VALUES (4, '0104', '23-SEP-2026', 'UPI', 'Success', 1500);  
1 row created.  
  
SQL> INSERT INTO Payment VALUES (5, '0105', '24-SEP-2026', 'Card', 'Failed', 2750);  
1 row created.
```

Insert Payment Records

```
SQL> SELECT * FROM PAYMENT;
```

PAYMENT_ID	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
TRANSACTION_AMOUNT				
1	0101	20-SEP-26	UPI	Success
	2500			
2	0102	21-SEP-26	Card	Success
	1800			
3	0103	22-SEP-26	Cash	Pending
	3200			
PAYMENT_ID	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
TRANSACTION_AMOUNT				
4	0104	23-SEP-26	UPI	Success
	1500			
5	0105	24-SEP-26	Card	Failed
	2750			

6. Display All Successful Payments

```
SELECT * FROM Payment WHERE Payment_Status = 'Success';
```

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Success';
```

PAYMENT_ID	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
TRANSACTION_AMOUNT				
1	0101	20-SEP-26	UPI	Success
	2500			
2	0102	21-SEP-26	Card	Success
	1800			
4	0104	23-SEP-26	UPI	Success
	1500			

7. Find All Failed Transactions

```
SELECT * FROM Payment WHERE Payment_Status = 'Failed';
```

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
5	0105	24-SEP-26	Card	Failed
	2750			

8. Count Total Successful and Failed Payments

```
SELECT Payment_Status, COUNT(*) AS Total_Payments FROM Payment WHERE Payment_Status IN ('Success', 'Failed') GROUP BY Payment_Status;
```

```
SQL> SELECT Payment_Status, COUNT(*) AS Total_Payments FROM Payment WHERE Payment_Status IN ('Success', 'Failed') GROUP BY Payment_Status;
```

PAYMENT_STATUS	TOTAL_PAYMENTS
Failed	1
Success	3

9. Update Failed Payments After Retry

```
UPDATE Payment SET Payment_Status = 'Success' WHERE Payment_Status = 'Failed';
```

```
SQL> UPDATE Payment SET Payment_Status = 'Success' WHERE Payment_Status = 'Failed';
```

1 row updated.

10. Identify Pending Transactions

```
SELECT * FROM Payment WHERE Payment_Status = 'Pending';
```

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Pending';
```

PAYMENT_ID	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
3	0103	22-SEP-26	Cash	Pending
	3200			

REPORT-1 (Payment Mode Analysis)

Find:

Number of transactions using UPI.

Number of card payments.

Most preferred payment method.

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE PAYMENT_MODE ='UPI';
```

```
UPI_TRANSACTIONS
```

```
-----  
2
```

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM PAYMENT WHERE PAYMENT_MODE ='Card';
```

```
CARD_PAYMENTS
```

```
-----  
2
```

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;
```

```
PAYMENT_MODE      TRANSACTION_COUNT
```

```
-----  
Card                2  
UPI                 2  
Cash                1
```

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM payment GROUP BY PAYMENT_MODE);
```

```
PAYMENT_MODE      TRANSACTION_COUNT
```

```
-----  
Card                2  
UPI                 2
```

Report 2: Revenue Analysis

Find:

Total revenue generated.

Revenue by payment method.

Average transaction amount.

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM payment GROUP BY PAYMENT_MODE);
```

PAYMENT_MODE	TRANSACTION_COUNT
Card	2
UPI	2

```
SQL> SELECT PAYMENT_MODE, SUM(TRANSACTION_AMOUNT) AS REVENUE
2 FROM PAYMENT
3 WHERE UPPER(PAYMENT_STATUS) = 'SUCCESS'
4 GROUP BY PAYMENT_MODE
5 ORDER BY REVENUE DESC;
```

PAYMENT_MODE	REVENUE
Card	4550
UPI	4000

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVERAGE_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS='Success';
```

AVERAGE_TRANSACTION_AMOUNT
2137.5

REPORT-3 Customer Payment History

Display:

Customer name

Order ID

Payment mode

Amount

Payment status

```
SQL> SELECT C.CUSTOMER_NAME,O.ORDER_ID,O.ORDER_DATE,P.PAYMENT_DATE,P.PAYMENT_MODE,P.TRANSACTION_AMOUNT AS AMOUNT,P.PAYMENT_STATUS FROM CUSTOMER C JOIN ORDER
S O ON C.CUSTOMER_ID=O.CUSTOMER_ID JOIN PAYMENT P ON O.ORDER_ID=P.ORDER_ID ORDER BY P.PAYMENT_DATE;
```

CUSTOMER_NAME		ORDER_I	ORDER_DAT	PAYMENT_D	PAYMENT_MODE
AMOUNT	PAYMENT_STATUS				
Devi	2500 Success	0101	01-SEP-26	20-SEP-26	UPI
Radhi	1800 Success	0102	02-SEP-26	21-SEP-26	Card
Ajay	3200 Pending	0103	03-SEP-26	22-SEP-26	Cash
Vijay	1500 Success	0104	04-SEP-26	23-SEP-26	UPI
Abi	2750 Success	0105	05-SEP-26	24-SEP-26	Card

Payment Analysis:

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT,SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE,ROUND(AVG(TRANSACTION_AMOUNT),2) AS AVG_TRANSACTION FROM PAY
MENT WHERE PAYMENT_STATUS='Success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
Card	2	4550	2275
UPI	2	4000	2000

11. Conclusion

The Payment Entity was successfully created and implemented using SQL. Primary key, foreign key and check constraints were applied to maintain data integrity. Payment records were inserted and analyzed using SQL